Lab Final

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

1 Lab Final

1. List films whose replacement cost is higher than the average replacement cost. Show title and replacement cost.

```
SELECT
    title,
    replacement_cost
FROM film
WHERE replacement_cost > (
    SELECT AVG(replacement_cost)
    FROM film
)
ORDER BY replacement_cost DESC;
```

2. List films that were never rented. Show film id and title.

```
SELECT f.film_id, f.title
FROM film f
LEFT JOIN inventory i ON i.film_id = f.film_id
LEFT JOIN rental r ON r.inventory_id = i.inventory_id
WHERE r.rental_id IS NULL
ORDER BY f.film_id;
```

3. List customers who have rentals from store 1 and store 2. Show customer id and full name.

```
SELECT c.customer_id, c.first_name, c.last_name
FROM customer c
WHERE c.customer_id IN (
    SELECT r.customer_id
    FROM rental r
    JOIN inventory i ON i.inventory_id = r.inventory_id
    WHERE i.store_id = 1
)
AND c.customer_id IN (
    SELECT r.customer_id
    FROM rental r
    JOIN inventory i ON i.inventory_id = r.inventory_id
    WHERE i.store_id = 2
)
ORDER BY c.customer_id;
```

4. Lists all films that are currently rented, ordered the result by film_id.

```
SELECT f.film_id,
       f.title,
       r.rental_id
FROM rental r
JOIN inventory i ON i.inventory_id = r.inventory_id
JOIN film f ON f.film_id = i.film_id
WHERE r.return_date IS NULL
ORDER BY f.film_id, r.rental_id;
```

5. Find films that were rented by customers from the highest number of different countries. Show film title and country count.

```sql
SELECT f.title,
       COUNT(DISTINCT co.country_id) AS country_count
FROM film f
JOIN inventory i ON i.film_id = f.film_id
JOIN rental r ON r.inventory_id = i.inventory_id
JOIN customer c ON c.customer_id = r.customer_id
JOIN address a ON a.address_id = c.address_id
JOIN city ci ON ci.city_id = a.city_id
JOIN country co ON co.country_id = ci.country_id
GROUP BY f.film_id, f.title
ORDER BY country_count DESC
LIMIT 5;
```

6. Write an anonymous block that prints revenue per month. Print month and total revenue using a loop.

```sql
DO $$
DECLARE
    rec RECORD;
BEGIN
    FOR rec IN
        SELECT TO_CHAR(p.payment_date, 'YYYY-MM') AS month,
               SUM(p.amount) AS revenue
        FROM payment p
        GROUP BY TO_CHAR(p.payment_date, 'YYYY-MM')
        ORDER BY month
    LOOP
        RAISE NOTICE 'Month % -> Revenue %', rec.month, rec.revenue;
    END LOOP;
END $$;
```

7. Write an anonymous block that prints the top 3 staff members by total payment collected. Print staff id, full name, and total revenue using a loop.

```sql
DO $$
DECLARE
    rec RECORD;
    i INT := 0;
BEGIN
    FOR rec IN
        SELECT s.staff_id, s.first_name, s.last_name,
               SUM(p.amount) AS total_revenue
        FROM staff s
        JOIN payment p ON p.staff_id = s.staff_id
        GROUP BY s.staff_id, s.first_name, s.last_name
        ORDER BY total_revenue DESC, s.staff_id
    LOOP
        i := i + 1;
        EXIT WHEN i > 3;

        RAISE NOTICE '%) Staff %: % % (Revenue=%)',
            i, rec.staff_id, rec.first_name, rec.last_name, rec.total_revenue;
    END LOOP;
END $$;
```

8. Write a function that takes a film id and prints whether the film is more popular than average. Print the film title, its rental count, and the average rental count. (A film is considered more popular than average if its rental count is greater than the average rental count per film).

```
DO $$
DECLARE
    v_film_id INT := 1;
    v_title TEXT;
    v_count INT;
    v_avg NUMERIC;
BEGIN
    SELECT title
    INTO v_title
    FROM film
    WHERE film_id = v_film_id;

    IF NOT FOUND THEN
        RAISE NOTICE 'Film id % not found', v_film_id;
        RETURN;
    END IF;

    SELECT COUNT(r.rental_id)
    INTO v_count
    FROM inventory i
    JOIN rental r ON r.inventory_id = i.inventory_id
    WHERE i.film_id = v_film_id;

    SELECT AVG(cnt)
    INTO v_avg
    FROM (
        SELECT COUNT(r2.rental_id) AS cnt
        FROM inventory i2
        JOIN rental r2 ON r2.inventory_id = i2.inventory_id
        GROUP BY i2.film_id
    ) sub;

    IF v_count > v_avg THEN
        RAISE NOTICE 'Film: % (id=%) Rentals=% Avg=% -> ABOVE average',
            v_title, v_film_id, v_count, ROUND(v_avg, 2);
    ELSE
        RAISE NOTICE 'Film: % (id=%) Rentals=% Avg=% -> NOT above average',
            v_title, v_film_id, v_count, ROUND(v_avg, 2);
    END IF;
END $$;
```

9. Create a function `film_revenue_range` that returns the total revenue earned by a given film within a given date range.

```
CREATE OR REPLACE FUNCTION film_revenue_range(
    p_film_id INT,
    p_from DATE,
    p_to DATE
)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
```

```
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT COALESCE(SUM(p.amount), 0)
    INTO v_total
    FROM inventory i
    JOIN rental r ON r.inventory_id = i.inventory_id
    JOIN payment p ON p.rental_id = r.rental_id
    WHERE i.film_id = p_film_id
      AND p.payment_date::date BETWEEN p_from AND p_to;

    RETURN v_total;
END;
$$;
-- Invocation (example)
SELECT film_revenue_range(1, '2007-02-14', '2007-05-14');
```

10. Create a procedure `print_late_days` that takes a rental id and prints how many days late it is. If the value is negative, print 0.

```
CREATE OR REPLACE PROCEDURE print_late_days(p_rental_id INT)
LANGUAGE plpgsql
AS $$
DECLARE
    v_rental_date DATE;
    v_duration INT;
    v_late_days INT;
BEGIN
    SELECT r.rental_date::date, f.rental_duration
    INTO v_rental_date, v_duration
    FROM rental r
    JOIN inventory i ON i.inventory_id = r.inventory_id
    JOIN film f ON f.film_id = i.film_id
    WHERE r.rental_id = p_rental_id;

    IF NOT FOUND THEN
        RAISE NOTICE 'Rental % not found', p_rental_id;
        RETURN;
    END IF;

    v_late_days := (CURRENT_DATE - v_rental_date) - v_duration;
    IF v_late_days < 0 THEN
        v_late_days := 0;
    END IF;

    RAISE NOTICE 'Rental % late days = %', p_rental_id, v_late_days;
END;
$$;

-- Example:
CALL print_late_days(1);
```

2 ER Diagram


actor
actor_id INT * pk
first_name VARCHAR(45) *
last_name VARCHAR(45) *
last_update DATETIME *
language
language_id INT * pk
name CHAR(20) *
last_update DATETIME *
film
film_id INT * pk
title VARCHAR(255) *
description TEXT
release_year YEAR
language_id INT *
rental_duration SMALLINT *
rental_rate DECIMAL(4,2) *
length SMALLINT
replacement_cost DECIMAL(5,2) *
rating film_rating_enum
last_update DATETIME *
special_features TEXT
fulltext_col TEXT *
category
category_id INT * pk
name VARCHAR(25) *
last_update DATETIME *
country
country_id INT * pk
country VARCHAR(50) *
last_update DATETIME *
city
city_id INT * pk
city VARCHAR(50) *
country_id INT *
last_update DATETIME *
address
address_id INT * pk
address VARCHAR(50) *
address2 VARCHAR(50)
district VARCHAR(20) *
city_id INT *
postal_code VARCHAR(10)
phone VARCHAR(20) *
last_update DATETIME *
film_actor
actor_id INT *
film_id INT *
last_update DATETIME *
inventory
inventory_id INT * pk
film_id INT *
store_id INT *
last_update DATETIME *
film_category
film_id INT *
category_id INT *
last_update DATETIME *
customer
customer_id INT * pk
store_id INT *
first_name VARCHAR(45) *
last_name VARCHAR(45) *
email VARCHAR(50)
address_id INT *
activebool BOOLEAN *
create_date DATE *
last_update DATETIME
active INT
staff
staff_id INT * pk
first_name VARCHAR(45) *
last_name VARCHAR(45) *
address_id INT *
email VARCHAR(50)
store_id INT *
active BOOLEAN *
username VARCHAR(16) *
password VARCHAR(40)
last_update DATETIME *
picture BLOB
rental
rental_id INT * pk
rental_date DATETIME *
inventory_id INT *
customer_id INT *
return_date DATETIME
staff_id INT *
last_update DATETIME *
store
store_id INT * pk
manager_staff_id INT *
address_id INT *
last_update DATETIME *
payment
payment_id INT * pk
customer_id INT *
staff_id INT *
rental_id INT
amount DECIMAL(5,2) *
payment_date DATETIME *